

VOID//SHIFT

Plano de melhorias

Combate consistente, escolhas mais interessantes e uma arquitetura mais simples de evoluir.

Este relatório organiza a análise do estado atual do shooter espacial em Python/Pygame. O foco é melhorar os sistemas existentes, consolidar a implementação e transformar a identidade narrativa em decisões de gameplay.

Base da análise

A revisão considerou o código do runtime, o sistema de minibosses recém-integrado, os fluxos de combate e persistência, os workflows do GitHub e a conferência visual headless anterior.

Evidências e propostas: os problemas técnicos descritos foram identificados no código ou em diagnósticos pequenos. Sugestões de diversão, dificuldade e balanceamento precisam de playtest. A última suíte completa executada antes da análise passou em **412 testes**; isso não substitui testar uma partida completa e o instalador.

Ordem de trabalho recomendada

Etapa	Melhoria	Resultado esperado
1	Combo e política de dano	Pontuação e armas com regras previsíveis.
2	Proteção dos saves	Menor risco de perder progresso.
3	Tempo da simulação	Combate consistente quando os FPS variam.
4	Responsabilidades de Jogo	Menos acoplamento e mudanças mais localizadas.
5	Ritmo dos encontros	Pressão, recuperação e surpresa bem distribuídas.
6	Upgrades e cascos	Mais variedade estratégica e ligação com a lore.

Em paralelo: medir performance, ampliar a CI e validar a experiência no Windows. Esta é uma proposta de evolução; o relatório não representa implementação dessas melhorias.

Regras previsíveis na partida

1. Revisar combo e dano das armas

Observado: o combo aumenta ao disparar, mesmo sem acertar um alvo. No diagnóstico, 30 disparos sem alvos produziram combo 30 e multiplicador de **2×**. O comportamento recompensa manter o botão pressionado.

Proposta: aumentar o combo por acertos ou abates consecutivos, com tolerância para pequenos intervalos. Definir claramente quando ele expira, o que o interrompe e como essa informação aparece no HUD.

Íon e Gauss também podem atingir a mesma entidade em vários quadros. Um diagnóstico geométrico simples, com uma hitbox parada, mostrou:

Arma	Dano configurado	Quadros em colisão	Dano acumulado
Íon	5	14	70
Gauss	3	5	15

Os valores representam aquela configuração de teste; não são uma medição de dano por segundo em uma partida completa.

Dano contínuo pode ser intencional. A melhoria é tornar a regra explícita: laser causa dano por segundo; um projétil perfurante acerta cada alvo uma vez. Isso evita balanceamento dependente do tamanho do alvo ou da quantidade de quadros de sobreposição.

Código: `src/runtime/domain/entities/player.py:253`; `src/runtime/controllers/combat.py:207`;
`src/runtime/domain/entities/weapons.py`.

2. Separar simulação e renderização

Observado: o loop atualiza o mundo uma vez por quadro e contabiliza **1 / FPS**. Quando a renderização cai para 30 FPS, movimento, cooldowns e encontros também ficam mais lentos.

Proposta: usar simulação com passo fixo e acumulador de tempo, mantendo a renderização separada. Limitar as atualizações de recuperação evita que um atraso provoque uma sequência longa de trabalho acumulado.

As transições de dimensão devem virar um estado atualizado pelo loop principal. Hoje elas executam laços próprios sem processar eventos dentro desses laços, o que pode atrasar a resposta a fechar a janela ou a comandos do jogador.

Código: `src/runtime/controllers/loop.py:108`; `src/runtime/controllers/progression.py:173`.
Referência: [Pygame — Clock.tick e medição do tempo transcorrido](#).

Escolhas que mudam o jogo

3. Criar ritmo nos encontros

O spawner de minibosses resolve quando um encontro aparece, mas a decisão também pode considerar o que já está acontecendo na tela.

Proposta: um diretor de encontros simples. Cada inimigo custa uma quantidade de “ameaça”, e cada onda tem um orçamento. Durante um miniboss, esse orçamento diminui.

Isso permite alternar pressão, recuperação e surpresa, além de evitar combinações frustrantes: inversão de controles, muitos projéteis e zonas bloqueando a movimentação ao mesmo tempo.

4. Aprofundar upgrades e cascos

Várias melhorias atuais são incrementos de atributos. Acrescentar benefícios com custos cria decisões mais interessantes:

Escolha	Benefício	Custo
Reator instável	Mais dano	Menos vida máxima
Blindagem pesada	Maior resistência	Esquiva recarrega mais devagar
Memória distribuída	Drone auxiliar	Menor carga de especial
Motor de emergência	Esquiva extra	Menor potência dos disparos

A lore permite que os cascos tenham comportamentos próprios, mantendo a aparência como escolha separada. Um cargueiro pode ser resistente; um interceptador, ágil; um protótipo corporativo, poderoso mas instável.

Fragmentos de memória dos antigos pilotos podem acompanhar esses desbloqueios. A história passa a aparecer nas decisões do jogador, além dos textos.

5. Melhorar leitura visual e acessibilidade

Na conferência visual anterior, fundo, brilho e HUD disputavam atenção em alguns momentos. A prioridade visual deve ser: projéteis hostis, hitbox do jogador e avisos de ataque.

- Reduzir o contraste do fundo durante encontros intensos.
- Diferenciar perigo por forma e animação, além da cor.
- Oferecer controle de intensidade de flashes e tremor.
- Mostrar a hitbox ao segurar uma tecla de precisão.
- Ensinar esquiva, especial e pontos fracos em situações curtas de jogo.

Menos acoplamento, mais segurança

6. Reduzir a responsabilidade de Jogo

Observado: `src/runtime/application/core.py` tem **1.405 linhas**, incluindo janela, combate, equipamentos, pausa e desenho de telas. Os controladores ajudam, mas vários ainda recebem o objeto Jogo inteiro e alteram seus campos diretamente.

Proposta: fazer extrações graduais, com responsabilidades claras:

- **SessaoPartida:** entidades e dados temporários.
- **SistemaCombate:** colisões, dano e mortes.
- **Apresentador:** janela, escala e backend gráfico.
- **Telas:** desenho e entrada próprios de cada interface.

Consolidar também as implementações paralelas: `src/runtime/` executa a partida, enquanto `src/domain/` e a máquina de estados nova têm estruturas que ainda não participam desse fluxo. Cada migração deve substituir uma responsabilidade existente, acompanhada de testes de paridade.

7. Unificar dano e efeitos

Há caminhos separados para inimigos, bosses, minibosses, explosões e plasma. Cada inimigo novo aumenta a quantidade de lugares que precisam conhecer suas particularidades.

Proposta: produzir um resultado comum de dano contendo dano aplicado, bloqueio, parte atingida e morte. Som, partículas, pontuação e drops reagem a esse resultado.

Isso melhora também os minibosses recém-integrados: as estratégias já estão separadas, mas ainda dependem de métodos internos de Boss e de integração específica no controlador.

8. Proteger o progresso do jogador

Observado: o save abre o arquivo diretamente com modo “w”. Uma interrupção durante a escrita pode deixar o JSON incompleto.

Proposta: gravar em um arquivo temporário na mesma pasta, validar o conteúdo e substituir o anterior, mantendo um backup recuperável. A biblioteca padrão oferece `os.replace` para a substituição.

Definir também um único responsável pelo saldo: atualmente loja, progresso e moedas da partida precisam ser sincronizados.

Código: `src/runtime/infrastructure/persistence/save_system.py:172`.

Referência: [Python — os.replace](#).

Medir e testar o jogo distribuído

9. Fortalecer testes e distribuição

A última suíte executada passou em 412 testes, mas o workflow de qualidade aplica Ruff a apenas três arquivos.

Proposta: ampliar gradualmente essa verificação, adicionar testes no Windows e validar o executável instalado iniciando e encerrando corretamente.

Há inconsistências de empacotamento que merecem atenção:

- requirements.txt usa pygame-ce, enquanto pyproject.toml usa pygame.
- O pacote inclui apenas src*, mas o menu ainda importa game.phase_select.
- É preciso validar a instalação fora do diretório do repositório, onde imports e recursos não podem depender dos arquivos da cópia de desenvolvimento.

Código: .github/workflows/quality.yml:16; pyproject.toml; requirements.txt; src/runtime/presentation/menu.py:22.

10. Medir antes de otimizar

Ainda existem alocações de superfícies grandes durante o desenho, inclusive em sombras do estilo original. Medir esses custos deve vir antes de introduzir estruturas mais complexas.

Separar também o RNG visual do RNG de combate: atualmente vários efeitos e decisões de gameplay compartilham sorteios globais. A separação facilita reproduzir problemas e comparar partidas.

Medida local	Decisão que ela ajuda a tomar
Tempo de quadro e quantidade de projéteis	Localizar quedas de desempenho em situações reais.
Duração dos encontros	Ajustar HP, intervalos e janelas de vulnerabilidade.
Causa das mortes	Identificar ataques confusos e combinações excessivas.
Armas e upgrades escolhidos	Verificar alternativas dominantes ou pouco úteis.
Moedas por partida	Ajustar recompensas e ritmo de desbloqueio.

Primeiro ciclo sugerido: corrigir combo e política de dano; proteger saves; estabilizar o tempo da simulação. Depois, extrair responsabilidades de Jogo, ajustar o ritmo dos encontros e aprofundar upgrades e cascos.

Os caminhos citados são relativos à raiz do repositório. Quantidade de linhas, testes e diagnósticos descrevem o estado revisado em 19/09/2026.